

Mining Software Repository to Identify Crosscutting Concerns Using Combined Techniques

Ingrid Marçal, Rogério Eduardo Garcia, Ronaldo C. M. Correia, Celso Olivete Junior
São Paulo State University – UNESP
College of Science and Technology
in.marcal@gmail.com, {rogerio,ronaldo,olivete}@fct.unesp.br

Abstract

Modularization is a goal difficult to achieve in software development. Aspect mining aims to identify crosscutting concerns in non aspect oriented software allowing organizing them into aspects and, thus, improve the modularization. Several techniques have been proposed to identify crosscutting concerns from software repositories, usually by analyzing static data or execution traces. In this paper present an approach on mining crosscutting concerns using combined results from two techniques: Frequent Closed Itemset Mining and Commit Frequency Analysis, which is presented in this paper as well. We evaluated our approach and observed that combining both techniques lead to better sets of crosscutting concerns candidates.

Keywords: code evolution, aspect mining, mining software repositories, software maintenance.

1 Introduction

Modularization is a goal difficult to achieve in software development. Some requirements, named crosscutting concerns (CC) [10], cannot be clearly mapped to isolated units of code, and their implementation tend to be scattered. Thus, modifying one piece of functionality might require changing scattered code leading to maintenance problems (e.g., entanglement, scattering and lack of legibility) [10]. Aspect Oriented Programming [10] offer mechanisms to mitigate problems caused by CC in software systems [13], allowing refactoring these CC into aspects. However, before refactoring process, it is necessary to identify existing CC in software artifacts (aspect mining).

Software analysis can be *static*, *dynamic* and *historical* [4]. In this paper we focus on historical analysis of data extracted from software repositories, what allows us to focus on how an artifact was modified over time, when it was changed, why it was changed, who changed it, and what

change had another as a side effect (changed together). Canfora et al. [4] pointed out that mining software repositories opens challenging research directions. Breu and Zimmermann [2] argue that CC might emerge along the software development, which means that some of them are no longer identified by static or dynamic analysis. Mulder and Zaidman [12] focused on mining items that are frequently changed together using a technique named *frequent closed itemset mining* [17], but they pointed out that it is necessary to improve the results, i.e., by combining with other.

Our main interest is to obtain results from different mining aspect techniques using the historical perspective of software development and investigate whether their combination improves their individual results. For this work, each source file (file name) in software repository is considered an item and each revision (version committed) a transaction. Our approach is based on how frequently source files are committed (*Commit Frequency Analysis*) and how frequently they are committed together (*Frequent Closed Itemset Mining*). We suggest filters to refine the results and evaluate the proposed approach using the *JHotDraw* project repository, which allow comparing results with other approaches in literature. Finally, we combine results from both mentioned techniques and observed an improvement on their individual results.

To present our approach, this paper is organized as follow: Section 2 presents a review of related works and the background required to analyze results; Section 4 shows the methodology for the experiments and the investigation; we present a discussion about our results in Section 5; at last, Section 6 presents final remarks about strengths and weaknesses of our approach, and suggestions to further works.

2 Background and Related Work

Object-Oriented Programming (*OOP*) is a paradigm that aids Software Engineering, since the underlying object model provides a better fit to real domain problems, but

OOP techniques are not sufficient to clearly capture important design decisions about different concerns implemented in a software system [10, 13]. Aspect-Oriented Program (AOP) tackles this so-called *crosscutting concerns* (CC) problem supporting the definition of implementation units (*aspects*) that cut across the system units, composing the expected behavior [10].

Aspect mining aims at creating and studying new methods and tools to identify CCs present in non aspect oriented software, thus they can be reorganized into aspects. Breu et al. [2] research opened a new perspective for aspect mining exploring software repositories to search for CCs. When applied to software repositories, aspect mining can be done basically in three steps: data acquisition, processing and presentation [12]. Data acquisition and presentation aim at collecting data from software repositories and presenting results, respectively. The core consists in applying a mining algorithm to process acquired data in order to extract and derive information. There are techniques and algorithms available in literature to process data from software repositories [2, 5, 14, 15]. Software repositories can be used for aspect mining purpose using different approaches, and in this paper we explore *Frequent Closed Itemset Mining* [15].

Mulder and Zaidman [12] applied *Frequent Closed Itemset Mining* in software repositories. *Frequent Closed Itemset Mining* is a constrained *Frequent Itemset Mining*, which is usually applied to discover association rules [3]. *Frequent Itemset Mining* describes the frequency of simultaneous appearance of related elements, using the following definitions summarized from [12].

2.1 Definitions for Frequent Itemset Mining

Let $I = \{I_1, I_2, \dots, I_m\}$ a set of items, $X \subseteq I$ an *itemset*, D a data base with a set $D = \{t_1, t_2, \dots, t_n\}$ of transactions, where $t_i = \{I_{i1}, I_{i2}, \dots, I_{ik}\} \mid I_{ij} \in I$. Let also, $t(X)$ a set of transactions that contains an *itemset* X , so $t(X) = \{Y \in D \mid Y \supseteq X\}$.

The *support* of an *itemset* X is defined as the ratio between the number of transactions on the data base that contains X and the total number of transactions on the data base, so

$$\text{support}(X) = \frac{|t(X)|}{|D|}$$

An *itemset* X is *frequent* if its *support* is greater than or equals to a given minimum *support*: $\text{support}(X) \geq \text{minsupport}$. The set of all frequent *itemsets* is denoted by FI and is a subset of all possible *itemsets* generated by I : $FI \subseteq 2^I$ [1][8].

We are not interested in subsets with the same *support* of their superset – they are considered irrelevant due to redundant information. In order to ignore irrelevant *itemsets* we have to look for frequent closed *itemsets*. A frequent

itemset is considered *closed* if there is no supersets with the same *support* [15, 12]. Formally, an *itemset* X is closed if: $I(t(X)) = X$, where $I(S) = \bigcap_{T \in S} T$ and $T, S \subseteq D$.

Frequent closed itemsets integrates a larger set of frequent items. Let FCI the set of all frequent closed *itemsets*, so: $FCI \subseteq FI$.

To measure the results obtained, the most common metrics in data extraction algorithms are *precision* and *recall*. *Precision* (P) is defined as the ratio of items predicted correctly and the total number of items predicted. *Recall* is defined as the ratio of items predicted correctly and the total number of items that can be selected. The *F1-measure* is a combination of *Precision* and *Recall*, defined as:

$$F1 = \frac{2(P \times R)}{P + R}$$

As result, $F1$ assumes values ranging from zero to one, where 0 is the worst case. For aspect mining purposes, on *Precision* metric *items predicted correctly* refers to the number of files confirmed as a CC, and *the total number of items predicted* is the number of items that composes the final result. Finally, on *Recall* metric the *total number of items that can be selected* refers to total number of known CCs.

Mulder [12] applied *Frequent Closed Itemset Mining* in two granularity levels: file-level and method-level. For file-level only file names were considered and for method level the author performed syntactic analysis on each file, to identify added and modified methods. It was observed that applying *Frequent Closed Itemset Mining* results in low precision, indicated by low F1-value on final results. The F1-value is provided by the F1-measure.

3 Commit Frequency Analysis

We have made some assumptions to propose the *Commit Frequency Analysis*. Kiczales et al. [10] have stated that: “In general, whenever two properties being programmed must compose differently and yet be coordinated, we say that they cross-cut each other. Because GP¹ languages provide only one composition mechanism, the programmer must do the co-composition manually, leading to complexity and tangling in the code”. Such “co-composition manually” is done using the mechanisms offered by languages, that is, methods and their callings or spread code across multiple source files.

Zimmermann et al. [17] pointed out that “every change in a method implies a change in the enclosing class”, what supports the rationale for searching changes in the enclosing files or packages to identify patterns like ‘this change implies another change in this package’. Instead of method

¹Generalized-Procedure languages for the purposes of the discussion, which is simpler to focus on what is common across several programming languages, including Object Oriented ones.

level, such search at file level might be less precise in locating modifications, but provide higher confidence [17]. Also, they pointed out that “the revision history can also tell us which parts of the system are coupled by common changes or co-changes”. Other evolutionary couplings and patterns are presented and discussed by Kagdi et al. [9]. Additionally, Van Rysselberghe and Demeyer [16] have introduced the concept of *Frequently Applied Changes* (FACs), defined as changes occurring multiple times in the system version history. They analyzed source code changes and suggested that FACs may be used to identify recurring change patterns and, in turn, identification of refactorings.

Based on that, we propose the *Commit Frequency Analysis* technique and its metric to analyze potential CCs. We proposed a top-down approach focusing on files (classes). The rationale considers each commit as a group of files with some coupling degree, and coupled files (classes) might be considered as CC candidates for further analysis. So, we do not focus on creating groups of related files (classes), we already have grouped files and focus on eliminated files (classes) that can be discarded as candidate to aspect. As metric, *Commit Frequency* represents the number of commits across multiples versions.

The approach proposed consists in a sequence of steps: 1) Compute the *Commit Frequency* (CF) for each file; 2) Discard files with CF equals to one, that are isolated commits (only groups of files must be considered as potential candidates); 3) Perform statistical analysis to obtain the shape of distribution and descriptive data; 4) Define an interval of interesting for analyzing by establishing minimal and maximal thresholds; 5) Apply those thresholds and obtain a list of files with CC candidates; 6) Obtain the list of versions where each file was updated; 7) Each file versions list is manually analyzed to verify whether there is a concern that matches; 8) Compute the F1-value for each group and, then, the average of F1-value to obtain an overall score. We have used a heuristic to define the interval of interesting that is presented in the following, along the application and results obtained.

4 Methodology

The framework proposed by Marin et al. [11] was used to state our approach on applying individually both techniques and combining them. Thus, our approach fits in the following: **Search goal**: CCs represented by files that are frequently and simultaneously modified – we do not restrict our investigation to unique sorts of concerns (e.g. *Consistent Behavior* and *Contract Enforcement* [11]) because our intent is investigate all possible resultant sort of concerns; **Results Format**: A list with file names that are frequently and simultaneously modified – this behavior is a characteristic of identifying CCs candidates; **Results Interpreta-**

tion: The resultant file names are entities names containing methods that are CC candidates; **Validation Metrics**: we manually compare our results to known CCs sets for the subject analyzed, then, the *F1-measure* is used to precise how the results match with these known set of CCs.

The approach proposed is depicted in Figure 1. Each step is supported by modules implemented in our own mining tool. We used SVNKit library to develop a module for acquiring data from software repository (**Data Acquisition**). Hence, we collected all entries from the history of development database that consists in: revision identification number, date, log message and the names of entities modified, added or excluded. Data collected from software repository are stored using Apache Commons Collection (hash table) using a format accepted by the algorithms developed (**Pre-processing**). As **Processing**, the LCM algorithm [15] was chosen to apply *Frequent Closed Itemset Mining* due its high performance with large amount of data; also, we created an algorithm to perform the *Commit Frequency Analysis*. Regarding the **Filtering**, a filter to *Data Acquisition* was defined, and the data set obtained has been used on both techniques; also, other filters were used on each technique, and their parameters are discussed in the following. After the experiments with *Frequent Closed Itemset Mining* and *Commit Frequency Analysis*, we combine results from both (**Combining**). Thus, it is possible to analyze whether combining the results from both techniques improves their individual results. Finally, the results are presented to the user in a list of entities names representing the CCs candidates identified (**Presentation**).

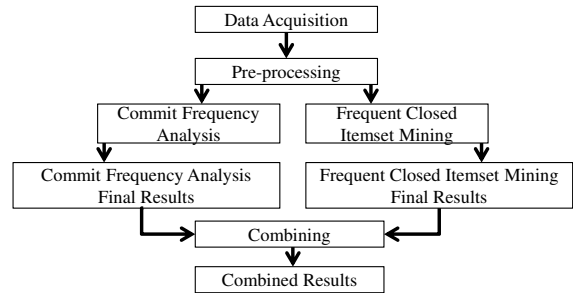


Figure 1: Schema proposed to combine results

5 Evaluating the approach

To validate the proposed approach and discuss the results obtained, it is necessary to use known CCs to compare to the CCs candidates obtained. However, there are few known crosscutting concerns documented and publicly available, which makes difficult to compare results. Marin et al. [11] proposed a web forum² where aspect mining researchers

²<http://sweri.tudelft.nl/amr/>

can exchange and discuss aspect candidates found in (open source) software systems. Despite their effort, only their own results are available there, obtained mainly from fan-in analysis. Therefore, our validation and discussion are based on information found in aforementioned forum and on a list of known CCs from other works [2, 5, 14]. Also, Frank Mulder kindly provided us with his results (list of CCs). The *JHotDraw* – a java framework to support graphical applications development – has been used in aspect mining literature. We chosen it to enable the validation of results by comparing to aspect mining results from previous works. All concerns identified by Ceccato *et al.* [14, 5], Marin *et al.* [11] and Mulder [12] applying *Dynamic Analysis*, *Identifier Analysis*, *Fan-in analysis* and *Frequent Closed Itemset Mining*, respectively, were compiled and are presented in Table 1.

Table 1: Known CC compiled from literature

Add text	Add URL to figure
Bring to front	Command executability
Connect figures	Connect text
Manage figure changed event	Figure update
Get attribute	Move figure
Manage handles	Manage figures outside drawing
Set Attribute	Send to back
Undo	Manage view rectangle
Visitor	Adapter
Consistent Behavior	Composite
Decorator	Exception Handling
Command	Observer
Persistence	Activation
Command Execution	Producer-Consumer
Event Handling	File Handling
Handling Mouse Events	Area Tracking
Background Drawing	Managing Display Boxes
Chopping Figures	Clearing Figures
Finding Figures	Color Choosing
Constraining Points	Figure Inclusion
Menu Handling	Managing Views
Font Handling	Image Handling
Create Drawings	Desktop Management
Invocation	Manipulating Figure Handles
Finding Connection	Dealing With Selections
Inserting Figures	Layout Calculation
Moving Figures	Handle and Figure Enumeration
Resource Management	Performing Actions
Iterating Over Collections	Working With Maps

5.1 Experiments and Results Discussion using FREQUENT CLOSED ITEMSET MINING

The first step (*Data Acquisition*) was executed to extract data from repository. For that, we used a filter applied to establish the minimum interval between commits by author – we considered that developers usually commit related changes in different revisions in short period and, thus, in this work we decided consider such revisions as a single

commit to reduce noise on final results. After analyzing metadata about the commits, we empirically decided to use a threshold of 1700 seconds between commits performed by the same author. So, we obtained a data set of 15747 files across 660 versions committed.

We have run several experiments applying *Frequent Closed Itemset Mining* to acquired data, using two parameters to compare their results. The first parameter is the minimum changeset size: very large and very small changesets can be ignored, since they rarely indicate CC [12]. And *Support*, the second parameter, is a measure that indicates the frequency of an *itemset*: high frequencies means possible presence of CCs. In this paper we related the frequency of an *itemset* with its size.

As initial experiment, *Frequent Closed Itemset Mining* was applied to *JHotDraw* repository considering only java source code – file names with *.java* extension, using *Support* = 2.0. As result, a set of 493 changesets was obtained, and its *F1-value* is 0.35. Next, the filters were applied to investigate their influences.

A set of experiments focused on investigating the relation between the precision and the changeset size. Thus, experiments were executed using changeset size as a constraint, and we established different changeset size empirically, keeping *Support* = 2.0. Initially, we used changeset size limited to 5, increasing this value at intervals of 5 for next 20 experiments running. We repeated a sequence of experiments using the initial changeset size limited to 10, and then increasing at intervals of 10. For experiments with changeset size less than 40, we observed low F1-values reaching the minimum value of 0.2. For experiments using changeset size equal to 40, the F1-value obtained is 0.449. Thus, we observed that the changeset size influences the results precision only for size values at the interval [2, 40]. Furthermore, experiments showed that few *itemsets* have high cardinality – usually *itemsets* with low cardinality are frequent – that is an important lesson learned used on evaluating *Commit Frequency Analysis*. It is important to note that the F1-value is not the same pointed out by Mulder and Zaidman, since they used JHotDraw version 5.4b1, and we considered all entries from JHotDraw repository.

Other experiments were conducted using different *Support* values – *Support* = 3.0 and *Support* = 4.0 –, but the results obtained were worse – even empty itemsets were obtained. Our best result (F1-value = 0.449) was better comparing to no filter result. However, we considered that it can be improved, especially taking into account that the F1-measure ranges at interval [0,1]. In order to explore alternatives to improve aspect mining precision, we investigate the application of *Commit Frequency Analysis* using software repository.

5.2 Experiments and Results Discussion using COMMIT FREQUENCY

To apply Commit Frequency Analysis we used the same data set previously obtained in *Data Acquisition* – 660 versions with 15747 files committed. The file list consists in 3358 files with no repetition across 660 versions. The *Commit Frequency* was computed for each file (step 1). We applied the filter established as *step 2*, selecting only files with CF greater than 1. At this point, 2236 files were selected across 212 versions.

The statistical analysis was performed according [7] (step 3). The measurements of location and variability are presented in Table 2. Also, we analyzed the frequency distribution and the normal probability curve, and we observe a concentration of commits at low CF values. At a first glance, we considered the minimal and maximal thresholds around the mean value – *mean $\pm$ standard deviation*. However, considering the distribution of normal probability plot (omitted due page restriction), we decided to be more restrictive, focus on interval $2 \leq CF < 10$ (step 4). It is important to note that there is no rule defined to choose the intervals, but statistical data description was used as parameter to make a decision. The heuristic is a greedy approach, trying to select more potential candidates, but we are aware that some CCs candidates can be out of the range, specially considering the Kurtosis value [6].

Table 2: Location and variability Measurements

Measure	Value
Mean	6,540698
Median	5
Standard Deviation	5,825551
Variance	33,93705
Kurtosis	17,3149

The filtering on *step 5* – considering CF-values on the interval $[2, 10[$ – resulted in 1838 files. So, we selected reviews where such files appear, and such reviews represent changesets with potential CCs candidates (step 6). We did not use any changeset filter. Each changeset was manually analyzed: we compared items implementation (file) to the concerns listed in Table 1 to verify whether the group items (files) can be refactored into aspect, or not (step 7). We considered the best matches to obtain the F1-value. To get an overall score for the groups, we take the average of these F1-values (step 8). As result, we get F1-value = 0.448.

One may observe that the F1-values obtained using both techniques were quite similar, what lead us to evaluate not only the measures, but also the data set obtained. *Frequent Closed Itemset* has generated data sets with less elements and, consequently, the F1-value reach such result by avoiding false positives, which increase the measures of precision and recall, but several CCs are ignored as well. The *Commit*

Frequency Analysis, on the other hand, has generated data sets with greater number of elements, but reach similar F1-value because it increases the number of false positive as well. The increasing of false positive requires (and wastes) time and effort on unnecessary analysis.

5.3 Combining Results

According to schema depicted in Figure 1, we aimed at improving individual results dealing with the limitations observed in *Commit Frequency Analysis*. We proposed to combine the results by taking the intersection of their individual results. For that, we used results from *Frequent Closed Itemset Mining* to compare to those files excluded from *Commit Frequency Analysis* due to filtering. We followed the steps: 1) Take the list of files excluded from previous *Commit Frequency Analysis*; 2) Compare the itemsets obtained from *Frequent Closed Itemset Mining*, creating a set of intersections S ; 3) For $s \in S$, obtain the version which updated s ; 4) Each file in version list must be manually analyzed to verify whether there is a concern that matches; 5) Compute the F1-value for each group and, then, the average of F1-value to obtain an overall score.

As first step, we filtered CF-values ≥ 10 getting 398 files. We compared those files to itemsets from *Frequent Closed Itemset Mining* – 44 files matched, but 33 was previously considered related to files that have CF-value less than 10 – grouped in some version and, therefore, considered in a changeset (step 2). So, 11 different files with CF-value in 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 31 should be submitted to a new *Commit Frequency Analysis*, what helps on dealing with the effect of filtering values in a distribution with high Kurtosis value. For that, a list of versions where those files were updated was created (step 3).

Each file in version list was manually analyzed, trying to use the best matching (step 4), resulting in 65 different CCs. The F1-value was computed (step 5) and overall F1-value was 0.590. This result seems better, but it was reached due to low cardinality on changeset (version) analyzed: the first *Commit Frequency Analysis* had average of changeset cardinality equal to 59.57, and for this analysis 9.47. Even with smaller number of CCs identified, the reduction of false positives had a positive influence to increase F1-value.

6 Conclusion

In this paper we have presented an approach for mining crosscutting concerns using results from two techniques: *Frequent Closed Itemset Mining* and *Commit Frequency Analysis*, which is presented in this paper as well. Both techniques deal with the same granularity level (files). *Commit Frequency Analysis* is described step by step, its

applications was presented, and we showed how to combine the results. In order to evaluate our approach, we chose a known open source project used on several researches. The results found in literature [2, 14, 5] were used as parameter to evaluate which crosscutting concern candidate would be classified as aspect. Also, this decision was based on practical judgments rather than concepts and principles, as found in literature, as well. We considered the best matches to not introduce any bias. It was observed that combining results leads to better sets of crosscutting concerns candidates. The greater F1-values and the increased number of crosscutting concerns identified are evidences of improvement.

Important observations were made about aspect mining using software repository and the chosen techniques for this paper, they are: a) the F1-values from both techniques are quite similar, but obtained from sets with different characteristics; b) the rate of false positives obtained along *Frequent Closed Itemset Mining* experiments is lower than ones obtained from *Commit Frequency Analysis*; c) we identified more crosscutting concerns applying *Commit Frequency Analysis* than applying *Frequent Closed Itemset Mining*; d) applying filters on both techniques improves their final result significantly; e) results from *Commit Frequency Analysis* are improved by combining *Frequent Closed Itemset Mining* results.

Our approach was validated with only one project software repository, which is a known benchmark on aspect mining research. However, we intent to conduct other experiments using other software projects repositories, for example, Eclipse³ and Tomcat⁴. Also, we intent to investigate how other techniques that deal with mining repositories at Method-Level Granularity can be used in a complementary way to our work at file-level analysis.

Acknowledgment

We would like to thank Frank Mulder for sending us a CSV file, sharing with us his results.

References

- [1] R. Agrawal, T. Imielinski, and A. Swami. Mining association rules between sets of items in large databases. In *SIGMOD '93: Proceedings of the 1993 ACM SIGMOD international conference on Management of data*, pages 207–216, New York, NY, USA, 1993. ACM Press.
- [2] S. Brey and T. Zimmermann. Mining aspects from version history. In *Proceedings of the 21st IEEE/ACM International Conference on Automated Software Engineering*, pages 221–230, Washington, DC, USA, Nov. 2006. IEEE Computer Society.
- [3] D. Burdick, M. Calimlim, and J. Gehrke. Mafia: A maximal frequent itemset algorithm for transactional databases. In D. Georgakopoulos and A. Buchmann, editors, *Proceedings. 17th International Conference on Data Engineering*, pages 443–452. IEEE Computer Society, 2001.
- [4] G. Canfora, M. Di Penta, and L. Cerulo. Achievements and challenges in software reverse engineering. *Commun. ACM*, 54(4):142–151, Apr. 2011.
- [5] M. Ceccato, M. Marin, K. Mens, L. Moonen, P. Tonella, and T. Tourwe. A qualitative comparison of three aspect mining techniques. In *Proceedings of the 13th International Workshop on Program Comprehension, IWPC '05*, pages 13–22, Washington, DC, USA, 2005. IEEE Computer Society.
- [6] L. T. DeCarlo. On the meaning and use of kurtosis. *Psychological Methods*, 2(3):292–307, 1997.
- [7] J. L. Devore. *Probability & Statistics For Engineering And The Sciences*. Books/Cole Cengage Learning, Boston, MA, 2012.
- [8] M. H. Dunham. *Data Mining: Introductory and Advanced Topics*. Prentice Hall, 2002.
- [9] H. Kagdi, M. L. Collard, and J. I. Maletic. A survey and taxonomy of approaches for mining software repositories in the context of software evolution. *Journal of Software Maintenance and Evolution: Research and Practice*, 19(2):77–131, March 2007.
- [10] G. Kiczales, J. Lamping, A. Mendhekar, C. Maeda, C. Lopes, J. marc Loingtier, and J. Irwin. Aspect-oriented programming. In *Proceedings of the European Conference on Object-Oriented Programming*, pages 220–242. SpringerVerlag, 1997.
- [11] M. Marin, L. Moonen, and A. van Deursen. A common framework for aspect mining based on crosscutting concern sorts. In *Proceedings of the 13th Working Conference on Reverse Engineering, WCRE'06*, pages 29–38, Washington, DC, USA, 2006. IEEE Computer Society.
- [12] F. Mulder and A. Zaidman. Identifying cross-cutting concerns using software repository mining. In *Proceedings of the Joint ERCIM Workshop on Software Evolution and International Workshop on Principles of Software Evolution, IWPSE-EVOL '10*, pages 23–32, New York, NY, USA, 2010. ACM.
- [13] E. F. Robert, E. Tzila, C. Siobhan, and A. Mehmet. *Aspect-Oriented Software Development*. John Wiley, 2005.
- [14] P. Tonella and M. Ceccato. Aspect mining through the formal concept analysis of execution traces. In *Proceedings of the 11th Working Conference on Reverse Engineering, WCRE'04*, pages 112–121, Washington, DC, USA, 2004. IEEE Computer Society.
- [15] T. Uno, T. Asai, Y. Uchida, and H. Arimura. Lcm: An efficient algorithm for enumerating frequent closed item sets. In *Proceedings of Workshop on Frequent itemset Mining Implementations*, 2003.
- [16] F. Van Rysselberghe and S. Demeyer. Mining version control systems for facts (frequently applied changes). In *International Workshop on Mining Software Repositories*, pages 48 – 52, Edinburgh, Scotland, UK, 2004. IEEE.
- [17] T. Zimmermann, P. Weissgerber, S. Diehl, and A. Zeller. Mining version histories to guide software changes. *IEEE Transactions on Software Engineering*, 31(6):429–445, June 2005.

³www.eclipse.org

⁴http://tomcat.apache.org